

Yet Another Conversational Humanoid Interactive Orchestrator: A Linearized Pipeline Architecture for Real-Time Embodied Conversational Agents

Yiyi Cai

Abstract

We present YACHIYO, a client-server architecture for real-time embodied conversational agents. The system models a complex multi-modal agent—comprising automatic speech recognition (ASR), large language model (LLM), retrieval-augmented generation (RAG), text-to-speech synthesis (TTS), and character animation—as a directed acyclic graph (DAG) of processing nodes, and linearizes it via topological sort into a sequential pipeline connected by thread-safe queues. We formalize *temporal consistency*—the guarantee that outputs respect causal, internal, and cancellation ordering—and show that linearization provides this guarantee with minimal implementation complexity. A dispatcher–receiver bracket recovers parallel execution for independent pipeline stages within the linear chain while preserving temporal consistency. A unified timestamp mechanism over WebSocket keeps the client and server synchronized, enabling streaming multi-modal output with low perceived latency. Each user owns an exclusive pipeline instance, while compute-heavy services run as shared standalone servers for independent scaling. We further introduce a WebRTC-based frame-level streaming mode that re-granularizes message boundaries from sentence-level to 20 ms frame-level, enabling continuous multi-modal output with sub-100ms synchronization. We describe the architecture, formalize the pipeline model, present latency measurements on a single NVIDIA RTX 5090 GPU, and discuss deployment experience.

1 Introduction

Problem. An embodied conversational agent must coordinate multiple processing stages in real time: speech recognition (ASR), language generation (LLM), information retrieval (RAG), speech synthesis (TTS), and character animation. Each stage has distinct computational requirements, latency characteristics, and software dependencies. The central challenge is: *how to orchestrate these heterogeneous stages into a system that is correct (outputs are ordered and synchronized), low-latency (the user perceives a responsive agent), scalable (multiple users can share expensive compute resources), and configurable (stages can be added, removed, or swapped without code changes)?*

Requirements. From first principles, a solution must satisfy:

1. **Temporal correctness:** outputs must arrive in the order implied by the dependency structure of the processing stages.
2. **Streaming:** downstream stages should begin processing as soon as partial results are available from upstream, rather than waiting for full completion.
3. **Client-server synchronization:** the rendering client and the processing server must agree on which output corresponds to which input, even under interruption and concurrency.
4. **Multi-modal synchronization:** audio, video, and metadata produced at different frame rates must be delivered to the client as a coherent, synchronized stream.

5. **Resource isolation with sharing:** each user needs an isolated execution context (conversation state, history), but expensive GPU models should be shared across users.
6. **Environment isolation:** different models may require incompatible software dependencies (e.g., different PyTorch versions for TTS vs. LLM).

Approach. We observe that the processing stages of a conversational agent form a directed acyclic graph (DAG) with a single entry and exit. We prove (Appendix A) that under mild assumptions, any such DAG can be *linearized*—reduced to a sequential chain of nodes connected by queues—without losing correctness. This linearization dramatically simplifies the execution model: each node is a thread consuming from one queue and producing to the next, with destination-based routing for skip-connections.

On top of this linearized pipeline, we layer:

- **Temporal consistency** (Section 4): we formalize three ordering properties (causal, internal, cancellation) and show that linearization guarantees all three with minimal implementation complexity.
- **Parallel branch execution** (Section 3.6): a dispatcher–receiver bracket recovers concurrent execution for independent nodes within the linear chain, using reverse-order emission and existing signal/forwarding mechanisms.
- **Intra-pipeline streaming:** the LLM streams sentences incrementally; each sentence immediately enters TTS, so the user hears audio while the LLM is still generating.
- **Timestamp-based synchronization:** a single timestamp per utterance propagates through the entire pipeline and back to the client, enabling correct ordering, stale-message filtering, and barge-in cancellation.
- **Frame-level re-granularization** (WebRTC mode): sentence-level messages are further split into 100 ms atomic groups of audio/video/data frames, synchronized via a GCD-based scheme.
- **Service-oriented compute:** heavy models run as standalone HTTP services with OpenAI-compatible APIs, called by lightweight per-user pipeline nodes.

We call the resulting system YACHIYO (**Y**et **A**nother **C**haracter Interaction **s**ystem **O**rchestrator). Existing frameworks address parts of this problem—LangChain [1] for LLM orchestration, LiveKit [4] for WebRTC transport—but none provides an integrated solution spanning pipeline linearization, temporal consistency, multi-modal streaming, client-server synchronization, and service-oriented scaling.

2 Related Work

Real-Time Conversational Agents. OpenAI’s GPT-4o Realtime API [2] provides end-to-end speech-to-speech generation but operates as a closed cloud service without user-controlled pipeline orchestration or custom model integration. OpenClaw [5], the most-starred open-source project on GitHub, is an autonomous AI agent platform that orchestrates multi-channel messaging, tool execution, and device integration through a WebSocket-based gateway. While it supports embodied interaction via paired device nodes, it focuses on general-purpose agent autonomy rather than real-time streaming pipeline orchestration for multi-modal conversational agents with character animation.

LLM Orchestration Frameworks. LangChain [1] provides composable abstractions for LLM chains including retrieval, tool use, and memory. LangGraph extends this with stateful graph-based workflows. However, these frameworks focus on text-level orchestration and do not address real-time streaming of audio, video, and animation data across client-server boundaries.

Node-Graph Processing Systems. ComfyUI [3] represents image and video generation workflows as visual DAGs where each node runs a model or transformation in-process. While flexible, this monolithic execution model requires all dependencies to coexist in one environment and does not support multi-user resource sharing or real-time streaming.

WebRTC for AI Agents. LiveKit Agents [4] and Daily Bots provide WebRTC infrastructure for connecting LLM-based agents to audio/video streams. These focus on transport-layer concerns and delegate pipeline logic to application code. Our system provides an integrated solution spanning pipeline orchestration, multi-modal synchronization, and real-time transport.

3 Pipeline Model

3.1 DAG Definition

A pipeline is a JSON configuration listing processing nodes in topological order. Each node has a unique, monotonically increasing ID, a processing function (resolved from a module registry), and a configuration block specifying its data routing (detailed in Section 3.4) and downstream connections. Variable names are prefixed with the producing node’s ID (e.g., `3_text`) to avoid namespace collisions. Node IDs need not be contiguous, allowing insertion without renumbering. See Appendix B for a complete example.

3.2 Why Linearization

The pipeline must guarantee *temporal consistency*: outputs corresponding to earlier inputs must arrive before outputs of later inputs, and outputs within a single utterance must arrive in production order. We formalize this property in Section 4; here we argue why linearization is the natural execution model.

Temporal consistency requires total ordering of messages. If messages can travel through the pipeline along multiple parallel paths, they may arrive at the output in a different order than they were produced. A merge point (where two branches rejoin) must resolve the ordering, which requires synchronization barriers—and barriers add complexity and latency.

A linear queue chain guarantees total order with minimal complexity. A chain of FIFO queues with single-threaded nodes preserves the order of messages by construction: if message m_1 enters before m_2 , then m_1 is dequeued and processed before m_2 at every node. No synchronization barriers or merge protocols are needed.

Alternative: full DAG parallel execution. Executing topologically parallel branches concurrently can improve throughput, but introduces merge points where messages from different branches must be reordered. Each merge point requires: (1) a synchronization barrier to wait for all incoming branches, (2) a merge protocol to establish the correct output order, and (3) handling of partial failures (one branch produces output while another is cancelled). For the same correctness guarantee, this is strictly more complex than the linear chain.

We prove in Appendix A that any DAG satisfying our assumptions can be linearized without loss of correctness.

Execution model. We restrict input DAGs to satisfy:

1. The graph has exactly one entry node and one exit node.
2. Node indices already form a valid topological ordering (edges always point from lower to higher IDs).

- Each node’s output depends only on messages it has already received.

[Figure placeholder: DAG linearization diagram. Left: DAG with skip-connections. Right: linearized queue chain $Q_0 \rightarrow N_0 \rightarrow Q_1 \rightarrow N_1 \rightarrow \dots \rightarrow Q_n \rightarrow \text{send_queue}$, with destination-based routing shown]

Figure 1: DAG linearization. The topological sort produces a linear chain of nodes connected by queues. Messages carry a **destination** field; intermediate nodes forward messages not addressed to them.

At initialization, the system arranges nodes in topological order and connects them via a chain of thread-safe queues:

$$Q_{\text{in}} \rightarrow N_0 \rightarrow Q_1 \rightarrow N_1 \rightarrow \dots \rightarrow Q_n \rightarrow Q_{\text{send}}$$

Each node runs in its own thread, blocking on its input queue and pushing results to its output queue. Formally, each node N_i executes:

$$N_i: \quad m \leftarrow Q_i.\text{dequeue}(); \quad \begin{cases} Q_{i+1}.\text{enqueue}(f_i(m)) & \text{if } m.\text{dest} = i \\ Q_{i+1}.\text{enqueue}(m) & \text{otherwise (forward)} \end{cases} \quad (1)$$

where f_i is node i ’s processing function, and $m.\text{dest}$ is the message’s destination tag.

Trade-off: serialization overhead. If two nodes N_a, N_b are topologically parallel (no path from one to the other), linearization forces a sequential order. A message m destined for N_b must wait behind N_a ’s processing of the preceding message. The added latency is bounded by:

$$\Delta L \leq \sum_{k \in \text{intervening}} L_k(m_{\text{prev}}) \quad (2)$$

where $L_k(m_{\text{prev}})$ is node k ’s processing time for the preceding message. For independent nodes that both process the same message, reordering does not reduce the combined latency—the total is always $\sum L_k$ regardless of ordering. We show in Section 3.6 that this overhead can be reduced to $\max(L_k)$ via a dispatcher–receiver bracket without abandoning the linear chain.

3.3 Streaming within the Linear Pipeline

[Figure placeholder: Streaming timeline. Horizontal axis = time. Rows: LLM (streaming sentences $s_1, s_2, s_3, \dots$), TTS (synthesizing s_1 while LLM produces s_2), Client (playing s_1 audio while TTS synthesizes s_2). Show overlap between stages.]

Figure 2: Streaming pipeline timeline. The LLM streams sentences incrementally; each sentence is immediately synthesized by TTS and played by the client, overlapping in time.

Linearization does not imply that each stage must complete before the next begins. The LLM node streams tokens incrementally; a *StreamCutter* module segments them into sentences

at punctuation boundaries. Each sentence is immediately pushed downstream as a separate pipeline message carrying the *same* timestamp as the original utterance. TTS processes each sentence independently, and the client plays the resulting audio in FIFO order—guaranteed by temporal consistency (Section 4).

We formalize the latency. Let the LLM generate K sentences $s_1, s_2, \dots, s_K$. Let L_k^{LLM} be the time to generate sentence s_k (from the end of s_{k-1}), and L_k^{TTS} be the time to synthesize s_k .

In **batch mode**, the client receives audio only after all stages complete:

$$L_{\text{batch}} = L_{\text{ASR}} + \sum_{k=1}^K L_k^{\text{LLM}} + \sum_{k=1}^K L_k^{\text{TTS}} + L_{\text{net}} \quad (3)$$

In **streaming mode**, the client receives the first audio segment as soon as the first sentence is processed:

$$L_{\text{stream}}^{\text{first}} = L_{\text{ASR}} + L_1^{\text{LLM}} + L_{\text{mid}} + L_1^{\text{TTS}} + L_{\text{net}} \quad (4)$$

where L_{mid} is the total processing time of intermediate nodes between LLM and TTS. When L_{mid} is small (e.g., a lightweight RAG lookup), it can be omitted from the approximations below; when it is large (e.g., motion generation), the dispatcher–receiver bracket (Section 3.6) can parallelize it with TTS.

Subsequent sentences overlap: while TTS synthesizes s_k , the LLM generates s_{k+1} . Defining $L_{K+1}^{\text{LLM}} = 0$ (no sentence after the last), the total time becomes:

$$L_{\text{stream}}^{\text{total}} = L_{\text{ASR}} + L_1^{\text{LLM}} + \sum_{k=1}^K \max(L_{k+1}^{\text{LLM}}, L_k^{\text{TTS}}) + L_{\text{net}} \quad (5)$$

When TTS dominates ($L_k^{\text{TTS}} \gg L_{k+1}^{\text{LLM}}$ for most k), the sum reduces to approximately $\sum_k L_k^{\text{TTS}}$, and total time approaches $L_{\text{ASR}} + L_1^{\text{LLM}} + \sum_k L_k^{\text{TTS}} + L_{\text{net}}$ —the LLM generation is almost entirely hidden behind TTS.

The speedup factor for first-output latency is:

$$S = \frac{L_{\text{batch}}}{L_{\text{stream}}^{\text{first}}} = \frac{L_{\text{ASR}} + \sum_k L_k^{\text{LLM}} + \sum_k L_k^{\text{TTS}} + L_{\text{net}}}{L_{\text{ASR}} + L_1^{\text{LLM}} + L_1^{\text{TTS}} + L_{\text{net}}} \quad (6)$$

3.4 Data Routing and Variable Mapping

Each inter-node message is a flat key-value object. Variable names are prefixed with the producing node’s ID to avoid collisions. Three routing mechanisms handle data flow without requiring nodes to know the full pipeline topology: (1) *input extraction* selects which upstream fields a node consumes, with ordered fallback sources; (2) *output mapping* maps processing results to prefixed target names; (3) *pass-through forwarding* carries metadata unchanged across nodes that do not consume it (e.g., the LLM’s action tag passes through TTS to reach the client). Pass-through is only applied when a node actually processes a message; forwarded messages (destination mismatch) are passed unchanged. A *destination* field enables skip-connections to non-adjacent nodes. See Appendix B for a concrete configuration example.

3.5 Signal Propagation: An Analogy with CPU Pipelines

Why signals are needed. A linearized pipeline processes *data* messages (audio, text, etc.) stage by stage. But some information is not data—it is *control*: “speech has started” (SoS), “speech has ended” (EoS), “voice activity detected” (vad_start/vad_end). Control information often needs to reach the client faster than data, because the client must update its state (e.g., begin the listening animation) before any processed data arrives.

Why signals should not overtake data. If control signals could freely overtake data messages in the queue, ordering guarantees would break: the client might receive EoS before the last audio chunk. This is analogous to the hazard problem in CPU instruction pipelines, where out-of-order execution of control instructions (branches) relative to data instructions causes incorrect results.

Our design: forwarding without processing. Instead of giving signals a separate fast path, we keep them in the *same* queue as data but exempt them from processing. When a signal arrives at a node that does not catch it, the node clears the signal’s destination and immediately forwards it to the next queue. The signal thus travels at the speed of queue forwarding ($\sim\mu\text{s}$ per hop) rather than the speed of data processing ($\sim\text{ms}$ per stage)—but only *after* it reaches the head of each queue. A signal does not jump ahead of messages already queued before it; it simply passes through each node’s processing step in negligible time. This is analogous to a CPU pipeline’s *bypass/forwarding* mechanism: the result is forwarded to where it is needed without waiting for the full stage to complete.

Each node declares which signals it catches. A signal emitted by any node will reach the client directly as long as no downstream node explicitly catches it—creating an express lane within the existing linear structure, without introducing a separate channel.

The one exception requiring out-of-band delivery is cancellation, discussed in Section 4.3.

3.6 Parallel Execution via Dispatcher–Receiver Bracket

Key observation. Linearization serializes independent nodes, adding latency ΔL (Section 3.2). However, this serialization is not inherent to the queue chain—it arises because each node must dequeue a message before the next can advance. If a message destined for a *later* node passes through an *earlier* node via forwarding (Equation 1, $\sim\mu\text{s}$) before the earlier node encounters its own message, both nodes process concurrently.

Mechanism: dispatcher and receiver. We introduce two meta-nodes that bracket a parallel region. Given k independent branch nodes $N_{b_1}, \dots, N_{b_k}$ (in topological order within the chain), the **dispatcher** and **receiver** operate as follows:

1. The dispatcher emits a **dispatch_start** signal (carrying upstream pass-through data) addressed to the receiver. This signal is forwarded through all branch nodes via destination-based routing.
2. The dispatcher emits k branch messages in *reverse topological order*: the message for N_{b_k} first, then $N_{b_{k-1}}$, down to N_{b_1} . Each message carries only the input fields required by its target branch.
3. The dispatcher emits a **dispatch_end** signal addressed to the receiver.

When N_{b_1} dequeues the branch messages, it first encounters the messages for $N_{b_2}, \dots, N_{b_k}$ (destinations do not match), forwarding each in μs . It then encounters its own message and begins processing. Meanwhile, the forwarded messages have already reached their respective nodes, which begin processing concurrently.

The receiver catches the **dispatch_start** and **dispatch_end** signals. Between them, it accumulates all arriving branch outputs. On **dispatch_end**, it merges the collected outputs with the pass-through data from **dispatch_start** and emits a single downstream message.

Why temporal consistency is preserved. FIFO ordering guarantees that all messages belonging to utterance u_i (with timestamp T_i) are dispatched, processed, and collected *before* any messages from u_{i+1} (with $T_{i+1} > T_i$). Specifically: (1) the dispatcher processes u_i before u_{i+1} (FIFO at the dispatcher’s input queue); (2) each branch node processes u_i ’s branch message before u_{i+1} ’s (FIFO at each branch’s queue); (3) the receiver collects u_i ’s **dispatch_end** before

u_{i+1} 's `dispatch_start` (FIFO at the receiver's queue). Causal and internal order are therefore maintained. Cancellation consistency is preserved by the same timestamp-based filtering as in Section 4.3.

Latency improvement. For k parallel branches with processing times $L_1, \dots, L_k$, the serialized latency is $\sum_{i=1}^k L_i$, while the parallel latency is $\max(L_1, \dots, L_k)$. We evaluate this improvement on a motion generation pipeline in Section 9.

4 Temporal Consistency

The pipeline's primary correctness requirement is *temporal consistency*: outputs delivered to the client must respect the ordering imposed by user utterances and the pipeline's production sequence. We now formalize this property and show how linearization guarantees it.

4.1 Definition

Let $\{u_1, u_2, \dots\}$ be a sequence of user utterances with timestamps $T(u_1) < T(u_2) < \dots$. For each utterance u_i , the pipeline produces an ordered sequence of output messages $O_i = (o_{i,1}, o_{i,2}, \dots, o_{i,K_i})$. A pipeline is *temporally consistent* if it satisfies three properties:

1. **Causal order.** If $T(u_i) < T(u_j)$, then all outputs of u_i are delivered to the client before any output of u_j —unless u_i has been cancelled.
2. **Internal order.** Within a single utterance u_i , outputs are delivered in production order: $o_{i,1}$ arrives before $o_{i,2}$ before $\dots$ before o_{i,K_i} .
3. **Cancellation consistency.** After a cancellation event `cancel( $T$ )` is issued, no message with timestamp $\leq T$ is subsequently delivered to the client.

4.2 Why Linearization Guarantees Temporal Consistency

Causal order. In a linearized pipeline, each node has exactly one input queue (FIFO) and processes messages one at a time in a single thread. If $T(u_i) < T(u_j)$, then u_i 's messages enter the pipeline before u_j 's. At every node in the chain, the FIFO queue guarantees that u_i 's messages are dequeued and processed before u_j 's. Therefore, u_i 's outputs arrive at the send queue before u_j 's.

Internal order. Within a single utterance, the LLM generates sentences $s_1, s_2, \dots, s_K$ sequentially, and the StreamCutter emits them in production order into the queue chain. Since every queue is FIFO and every node is single-threaded, s_1 's outputs are produced before s_2 's at every stage. The internal order is preserved end-to-end.

Linearization makes temporal consistency a structural guarantee rather than a runtime enforcement. The dispatcher–receiver bracket (Section 3.6) preserves this property by ensuring that all branch messages for u_i are dispatched, processed, and collected before any branch messages for u_{i+1} , as shown in the FIFO argument of Section 3.6.

4.3 Cancellation Consistency

Why cancellation needs out-of-band delivery. When the user interrupts (barge-in), every pipeline node must *immediately* stop processing stale messages. If the cancel signal traveled through the data queue, it would wait behind the very messages it intends to invalidate—defeating its purpose. This is analogous to a CPU pipeline *flush*: upon a branch misprediction, all in-flight instructions must be discarded simultaneously, not sequentially.

Design. Each node has a dedicated `cancel_queue` (parallel to its data queue), checked before each dequeue operation. When the client sends `cancel(T)`, the server broadcasts T to every node’s cancel queue. Each node updates its local `cancel_timestamp`; any subsequently dequeued message with timestamp $\leq \text{cancel_timestamp}$ is silently dropped.

The send queue performs a final filter: before transmitting any message to the client, it verifies that the message’s timestamp exceeds the current cancel timestamp. This provides a safety net for messages that were already past intermediate nodes when the cancel arrived.

Proof of no leakage. After `cancel(T)` is issued, consider any message m with $\text{timestamp}(m) \leq T$. At the moment the cancel is broadcast, m is in one of three states:

- **Waiting in a queue, not yet dequeued:** when the node dequeues m , it checks the cancel timestamp (updated via the cancel queue) and drops m without processing.
- **Currently being processed by node N_i :** the node cannot abort mid-computation, so it completes and enqueues its output $f_i(m)$ into Q_{i+1} . However, $f_i(m)$ inherits the same timestamp as m . When the next node N_{i+1} dequeues $f_i(m)$, it checks the cancel timestamp and drops it. If $f_i(m)$ propagates through multiple forwarding nodes before being caught, the send queue’s final filter guarantees it is not delivered.
- **Already in the send queue:** the send queue checks the cancel timestamp before each transmission and drops m .

In all three cases, m (or any output derived from m) is never delivered to the client after `cancel(T)`, satisfying cancellation consistency. $\square$

5 Client-Server Synchronization

5.1 Timestamp Mechanism

First-principles motivation. The causal origin of a conversational event is the client: the user spoke. Therefore, the timestamp that identifies the event should be assigned by the client at the moment of input. If the server assigned timestamps instead, variable network latency would corrupt the ordering: two utterances sent in quick succession might receive server-side timestamps in the wrong order, or the same order but with distorted intervals. By using the client’s wall clock, the timestamp reflects the true causal ordering of user actions.

Design. Every message carries a `timestamp` field, preferring the client-provided value (wall clock at input time); if absent, the server falls back to its own clock. The timestamp is set once at utterance start and propagated unchanged through all pipeline stages as a reserved field. The client uses it to associate responses with utterances, discard stale output, and maintain ordering under concurrency.

6 WebRTC: Frame-Level Streaming

6.1 Two Paradigms: Event-Driven vs. Frame-Driven

WebRTC is a real-time communication protocol that provides *tracks* (continuous media streams for audio and video) and a *DataChannel* (a reliable or unreliable channel for arbitrary application data). The two communication modes in YACHIYO reflect fundamentally different timing paradigms:

WebSocket: event-driven, client-timestamped. Messages are produced only on events (user speech, LLM output, TTS completion); between events the channel is silent. The timing origin is the client’s wall clock, propagated through the pipeline.

WebRTC: frame-driven, server-timestamped. Audio and video tracks produce frames at fixed rates (50 fps, 30 fps) regardless of activity, emitting silence when idle. The timing origin is the server’s monotonic frame clock.

Paradigm adapters. Two adapter nodes bridge the paradigms: AudioCollector assembles continuous microphone frames into a single WAV buffer on `vad_end` (frame $\rightarrow$ event); FrameSplitter splits TTS audio into 20 ms PCM frames (event $\rightarrow$ frame). The core pipeline remains purely event-driven.

6.2 Motivation: From Sentence-Level to Frame-Level

In WebSocket mode, the message boundary is a complete TTS segment ($\sim 0.5\text{--}1$ s of audio). The client receives a whole audio clip and then plays it, resulting in a perceptible “chunk” rhythm. WebRTC mode re-granularizes the output to frame-level:

- Audio: 20 ms PCM frames (960 samples at 48 kHz).
- Video: ~ 33 ms JPEG frames (30 fps).
- Data (text, action metadata): 50 ms slots (20 fps).

The output becomes a continuous stream rather than discrete chunks.

6.3 Pipeline Extension for WebRTC

Two adapter nodes are inserted at the boundaries of the pipeline:

- **AudioCollector** (head): collects incoming WebRTC audio frames between `vad_start` and `vad_end` signals, assembles them into a complete WAV buffer, and pushes it to the ASR node.
- **FrameSplitter** (tail): receives complete TTS audio output (WAV), resamples to WebRTC rate, splits into synchronized frame groups, and pushes them to the WebRTC tracks.

The core pipeline (ASR $\rightarrow$ LLM $\rightarrow$ RAG $\rightarrow$ TTS) remains unchanged. AudioCollector and FrameSplitter are *paradigm adapters*—they translate between event-driven and frame-driven data representations at the pipeline boundary, allowing the core pipeline to remain purely event-driven regardless of the transport protocol.

6.4 Multi-Modal Frame Synchronization

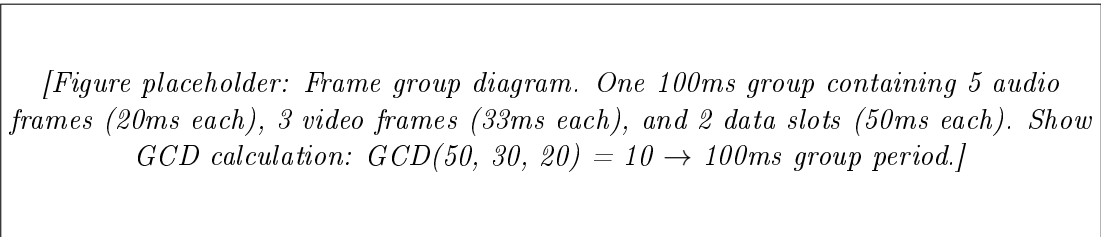


Figure 3: Multi-modal frame synchronization. Each 100 ms group bundles 5 audio frames, 3 video frames, and 2 data slots into an atomic synchronization unit.

Three modalities with different frame rates must remain synchronized:

- Audio: $R_a = 48000/960 = 50$ fps
- Video: $R_v = 30$ fps
- Data: $R_d = 20$ fps

Deriving the group period from first principles. We need the smallest period T_g such that each modality produces an *integer* number of frames within T_g . This requires $T_g \cdot R_a$, $T_g \cdot R_v$, and $T_g \cdot R_d$ to all be positive integers, i.e., $T_g = k / \gcd(R_a, R_v, R_d)$ for the smallest positive integer $k = 1$. Thus:

$$R_g = \gcd(R_a, R_v, R_d) = \gcd(50, 30, 20) = 10 \text{ Hz}, \quad T_g = \frac{1}{R_g} = 100 \text{ ms} \quad (7)$$

This is the *finest* common synchronization granularity—any shorter period would require fractional frames in at least one modality. Each group contains $n_a = R_a/R_g = 5$ audio frames, $n_v = R_v/R_g = 3$ video frames, and $n_d = R_d/R_g = 2$ data slots.

The group is the atomic synchronization unit: the client consumes one group per T_g , dispatching exactly n_a audio frames, n_v video frames, and n_d data slots to their respective output channels. Metadata (text, action, emotion) is placed in the first data slot of the first group for each TTS segment; remaining data slots are null. This guarantees that no modality drifts ahead of or behind others within a group.

6.5 Comparison: WebSocket vs. WebRTC Mode

Table 1: Comparison of communication modes.

Aspect	WebSocket	WebRTC
Message granularity	Sentence (~ 0.5 – 1 s)	Frame group (100 ms)
Output continuity	Discrete chunks	Continuous stream
Audio format	Base64 WAV	Raw PCM (20 ms)
Synchronization	Timestamp per message	Atomic frame groups
Use case	Web UI, simple clients	2D avatar, immersive

7 Resource Sharing and Scalability

7.1 Per-User Pipeline, Shared Services

First-principles motivation. Pipeline state consists of threads, queues, and per-user context (conversation history, session variables)—on the order of kilobytes per user. This is cheap to replicate, so each user gets a dedicated pipeline instance with full isolation. Model state, by contrast, consists of neural network weights loaded into GPU VRAM—on the order of gigabytes per model. Replicating model state per user is prohibitively expensive. The natural division is: replicate what is cheap (pipeline instances), share what is expensive (model services).

*[Figure placeholder: System architecture diagram showing Unity client ↔
WebSocket/WebRTC ↔ FastAPI server ↔ Standalone services (ASR, LLM, TTS,
VectorDB)]*

Figure 4: System architecture. The client communicates with the pipeline server via WebSocket or WebRTC. The pipeline server orchestrates per-user pipeline instances, each calling shared standalone model services via HTTP APIs.

Each user gets an exclusive pipeline instance (threads, queues, conversation state). Compute-heavy models are deployed as standalone HTTP services:

- ASR: Qwen3-ASR server (GPU, vLLM backend)
- LLM: vLLM server (GPU)
- TTS: Qwen3-TTS server (GPU, faster-qwen3-tts)
- Vector DB: BGE embedding + FAISS server (GPU/CPU)

Pipeline nodes are thin HTTP clients: they create an OpenAI SDK client, call the API, and process the response. Multiple users’ pipelines share the same model servers concurrently.

7.2 OpenAI-Compatible API Layer

Motivation. Pipeline nodes communicate with model services via HTTP, which provides environment isolation (each service runs in its own process with its own dependencies). The remaining design choice is the HTTP protocol: a custom API or an existing standard. The OpenAI API has become the de facto standard in the LLM ecosystem—every major model serving framework (vLLM, Ollama, TGI) already implements it, and client SDKs are freely available. Adopting this standard minimizes integration effort: swapping a local model for a cloud API (or vice versa) requires only changing the configuration file, with no code modification.

All standalone services expose OpenAI-compatible endpoints:

- ASR: `POST /v1/audio/transcriptions` (Whisper API format)
- LLM: `POST /v1/chat/completions` (Chat Completions API format)
- TTS: `POST /v1/audio/speech` (TTS API format)

Model selection is config-driven: each model has a JSON configuration file specifying `api_base`, `api_key`, `model`, and `extra` parameters. Each module makes an initialization call at startup to verify service availability and trigger any needed model loading (e.g., TTS voice model switching).

7.3 Comparison with Monolithic Execution

Systems like ComfyUI [3] execute all nodes within a single runtime, loading every model into the same process and GPU memory space. While this avoids inter-process communication overhead, it creates several constraints that conflict with the requirements of a multi-user, real-time conversational system:

1. **Resource sharing:** in a monolithic system, each session must load its own model instances or implement complex in-process model multiplexing. In our architecture, a single GPU-hosted model server serves many concurrent users transparently, with the serving framework (e.g., vLLM’s continuous batching) handling request scheduling.
2. **Independent scaling:** if the TTS service becomes the bottleneck, it can be replicated to additional GPUs without touching the pipeline or other services. In a monolithic system, scaling one component requires replicating the entire runtime.
3. **Environment isolation:** each service runs in its own environment (e.g., different PyTorch versions for TTS and LLM), and swapping a model from local to cloud is a configuration edit.
4. **Fault isolation:** a crash in one service does not bring down the entire system; other users’ pipelines continue operating.

The trade-off is HTTP overhead per call ($\sim 1\text{--}10\text{ ms}$), which is negligible when per-node inference time is $30\text{ ms--}1\text{ s}$.

8 Implementation

8.1 Technology Stack

The server is implemented in Python 3.12 using FastAPI and uvicorn for HTTP/WebSocket serving, aiortc for WebRTC, and the OpenAI Python SDK for model service calls. Each client follows the lifecycle: **register** $\rightarrow$ **init_pipeline** (from a JSON configuration) $\rightarrow$ **connect** (WebSocket or WebRTC) $\rightarrow$ pipeline runs $\rightarrow$ **unregister**. The pipeline is instantiated per-client with a dedicated thread per node and inter-node queues.

The client is built with Unity 2022 (C#) using WebSocket-Sharp and Unity WebRTC for communication, and the Animator controller with blend shapes for character animation. The system comprises ~ 15 pipeline module types, dynamically discovered via a module registry. All configuration is JSON-based: pipeline configs, dataset configs, and model configs. Deployment uses per-service conda environments with a startup script that launches services sequentially and verifies port readiness.

8.2 Client Architecture

The Unity client mirrors the server’s pipeline design: modules (VAD, recording, WebSocket communication, audio playback, animation) are connected in a sequential queue chain with the same destination-based routing, timestamp propagation, and cancel-queue mechanism. This symmetry means the client and server share the same correctness guarantees (temporal consistency, cancellation consistency) without additional coordination logic.

[Figure placeholder: State machine diagram. Four states: Idle, Ready, Listening, Answering. Transitions: Idle $\xrightarrow{\text{ready_start}}$ Ready $\xrightarrow{\text{listening_start}}$ Listening $\xrightarrow{\text{listening_end}}$ Answering $\xrightarrow{\text{answering_end}}$ Ready. Dashed arrow: any state $\xrightarrow{\text{cancel}}$ Idle.]

Figure 5: Client-side state machine governing agent behavior.

A four-state machine governs the interaction flow:

- **Idle**: character plays idle animation, awaiting activation.
- **Ready**: pre-listening countdown; times out to Idle if no input arrives.
- **Listening**: dual-threshold VAD detects speech; audio is recorded and sent to the server.
- **Answering**: the client plays back server responses (audio, text, animation) in FIFO order. Barge-in returns to Listening.

The state machine is purely client-side; the server does not track client state. For motion generation pipelines, the client includes a SMPLH motion player that converts received joint parameters into real-time skeletal animation via per-frame interpolation.

8.3 LLM Node: Tavern-Style Conversation Workflow

The LLM node internally implements a structured dialogue management workflow inspired by the SillyTavern character card format:

Multi-turn history is persisted per client. Character personality is defined in external “lore-book” files whose entries are activated by keyword or vector similarity and injected into the prompt at each turn. The LLM streams tokens, segmented into sentences by the StreamCutter; action tags are parsed for downstream animation mapping; tool calls are executed in a loop. This

workflow runs entirely within the LLM node—upstream and downstream nodes are unaware of it.

9 Evaluation

All experiments are conducted on a single workstation equipped with an NVIDIA GeForce RTX 5090 GPU (32 GB VRAM). All local services—Qwen3-ASR-0.6B (ASR via vLLM), Qwen3.5-9B-AWQ (LLM via vLLM), Qwen3-TTS-0.6B via faster-qwen3-tts (TTS), BGE-M3 (vector database)—run concurrently on this single GPU, occupying ~ 27 GB VRAM total. The test input is a 1.6-second Chinese speech clip.

9.1 Per-Stage and End-to-End Latency

Table 2: Per-stage and end-to-end latency: local pipeline vs. OpenAI API. All values in ms (mean $\pm$ std, 4 runs excluding warmup).

Stage	Local (RTX 5090)	OpenAI API
ASR	28 ± 1	886 ± 386
LLM (total generation)	290 ± 88	1252 ± 189
TTS (first sentence)	875 ± 323	1788 ± 454
Server first audio	1101 ± 331	3505 ± 860
E2E first audio (client)	1139 ± 347	3541 ± 855
E2E total	2228 ± 754	7264 ± 2927

The local pipeline achieves a first-audio latency of ~ 1.1 s, with TTS synthesis dominating (~ 0.9 s per sentence). The OpenAI API pipeline incurs ~ 3.5 s first-audio latency, primarily due to network round-trip times. The streaming architecture means that the total response time (~ 2.2 s local) includes overlapping computation: while TTS synthesizes sentence n , the LLM is already generating sentence $n+1$.

9.2 Multi-User Scalability

Table 3: Multi-user latency on a single RTX 5090 (local pipeline). All values in milliseconds.

Users	First Audio (avg)	First Audio (max)	Total (avg)
1	966	966	1372
2	2070	2072	3587
3	2878	2883	5427

When U concurrent users share services on a single GPU, latency depends on the serialization behavior of each service. The LLM server (vLLM) supports continuous batching and can partially parallelize concurrent requests, but the TTS server (Qwen3-TTS) processes requests sequentially. Modeling the TTS as the serialization bottleneck, the expected first-audio latency for user u (ordered by arrival) is approximately:

$$\mathbb{E}[L_u] \approx L_{\text{stream}}^{\text{first}} + (u - 1) \cdot L_{\text{bottleneck}} \quad (8)$$

where $L_{\text{bottleneck}}$ is the processing time of the slowest shared service (TTS in our case, ~ 0.8 s). The average across all users is:

$$\bar{L} \approx L_{\text{stream}}^{\text{first}} + \frac{U - 1}{2} \cdot L_{\text{bottleneck}} \quad (9)$$

Table 3 confirms this model: for 1–3 users, first-audio latency increases by ~ 950 ms per additional user, consistent with TTS serialization (~ 875 ms plus scheduling overhead). With R replicas of the bottleneck service, the effective capacity scales as $\bar{L} \approx L_{\text{stream}}^{\text{first}} + \frac{U-1}{2R} \cdot L_{\text{bottleneck}}$.

9.3 WebRTC Streaming

In WebRTC mode, the pipeline successfully delivers frame-level synchronized output over a 45-second test session:

- 901 DataChannel messages (one 100 ms frame group each)
- 2262 audio frames sent, 2259 received (48 kHz stereo)
- 1357 video frames sent and received (30 fps)
- ASR correctly transcribes the test input via the AudioCollector–ASR path
- Audio, video, and data remain synchronized throughout

The FrameSplitter overhead is negligible (< 1 ms per group), confirming that frame-level re-granularization does not add meaningful latency.

9.4 Configurability

The same pipeline framework supports multiple configurations without code changes:

Table 4: Example pipeline configurations.

Config	Pipeline
Conversation	ASR $\rightarrow$ LLM $\rightarrow$ RAG $\rightarrow$ TTS
Conversation (WebRTC)	AudioCollector $\rightarrow$ ASR $\rightarrow$ LLM $\rightarrow$ RAG $\rightarrow$ TTS $\rightarrow$ FrameSplitter
Conversation (cloud)	ASR ^{cloud} $\rightarrow$ LLM ^{cloud} $\rightarrow$ RAG $\rightarrow$ TTS ^{cloud}
Motion generation	ASR $\rightarrow$ LLM $\rightarrow$ MotionGen $\rightarrow$ TTS
Motion gen. (parallel)	ASR $\rightarrow$ LLM $\rightarrow$ Dispatch $\rightarrow$ MotionGen $\parallel$ TTS $\rightarrow$ Receive

Switching between local and cloud models requires only editing the model JSON configuration file (changing `api_base` and `api_key`).

9.5 Motion Generation and Parallel Execution

An alternative pipeline replaces RAG with a text-to-motion generation service (HY-Motion), which produces SMPLH body parameters from LLM action descriptions. The HY-Motion per-stage latency (mean $\pm$ std over 3 runs): prompt rewriting 127 ± 10 ms, text encoding 53 ± 1 ms, ODE solver (50 steps) 276 ± 5 ms, decode + SMPLH 9 ± 0 ms; end-to-end 884 ± 36 ms.

Since MotionGen and TTS are independent (both consume LLM output), they can be parallelized via the dispatcher–receiver bracket (Section 3.6):

Table 5: First-audio latency: sequential vs. parallel motion generation pipeline (mean $\pm$ std over 3 runs, single RTX 5090).

Configuration	First Audio (ms)
Sequential (MotionGen $\rightarrow$ TTS)	1649 ± 199
Parallel (MotionGen $\parallel$ TTS)	1536 ± 172
Improvement	113 ms ($\downarrow 6.9\%$)

The improvement (113 ms, 6.9%) is modest because the MotionGen service adds ~ 540 ms to the pipeline (sequential first audio 1649 ms vs. standard 1110 ms), but TTS (~ 875 ms) still

dominates the critical path in the parallel configuration. The theoretical maximum improvement equals the MotionGen latency (~ 540 ms), but is limited by the dispatcher–receiver overhead and the fact that both branches share GPU resources.

10 Discussion

Limitations. The dispatcher–receiver bracket (Section 3.6) supports fork–join parallelism at any point in the pipeline, but requires that parallel branches be independent (no data flow between branches within a bracket). Nested brackets (parallel regions within parallel regions) are possible in principle but have not been tested. The WebRTC FrameSplitter assumes fixed frame rates; adaptive rate adjustment would be needed for variable-bandwidth networks.

Future Work. Planned extensions include: (1) nested dispatcher–receiver brackets for multi-level parallelism; (2) vision input modality (camera $\rightarrow$ image understanding); (3) adaptive frame rate synchronization for WebRTC; (4) multi-GPU TTS serving for concurrent multi-character support.

11 Conclusion

We presented YACHIYO, a linearized pipeline architecture for real-time embodied conversational agents. The key contributions are:

1. A DAG-to-linear-chain transformation that ensures correct execution order while enabling streaming at every stage, reducing first-output latency from the sum of all stage latencies to the critical path of a single sentence.
2. A formal definition of *temporal consistency* (causal order, internal order, cancellation consistency) and a proof that the linearized pipeline guarantees all three properties.
3. A dispatcher–receiver bracket that recovers parallel execution within the linear chain by exploiting reverse-order emission and destination-based forwarding, while preserving temporal consistency.
4. Two levels of streaming granularity: sentence-level (WebSocket) and frame-level (WebRTC), with a GCD-based multi-modal synchronization mechanism for atomic 100 ms frame groups.
5. Timestamp-based client-server synchronization enabling correct ordering, stale message filtering, and clean barge-in cancellation.
6. Separation of per-user lightweight pipeline instances from shared GPU-hosted model services, connected via an OpenAI-compatible API layer that allows transparent swapping between local and cloud backends.

On a single NVIDIA RTX 5090 GPU hosting all services, the system achieves ~ 1.0 s first-audio latency for a fully local pipeline and supports up to 3 concurrent users within interactive latency bounds (< 3 s first audio).

References

- [1] Chase, H. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>, 2022.
- [2] OpenAI. GPT-4o and the Realtime API. <https://platform.openai.com/docs/guides/realtime>, 2024.
- [3] comfyanonymous. ComfyUI: A powerful and modular stable diffusion GUI and backend. <https://github.com/comfyanonymous/ComfyUI>, 2023.

- [4] LiveKit Inc. LiveKit Agents: Build real-time AI agents. <https://docs.livekit.io/agents/>, 2024.
- [5] OpenClaw. OpenClaw: Personal AI assistant you run on your own devices. <https://github.com/openclaw/openclaw>, 2025.

A Proof: DAG Linearization

We prove that the class of DAGs arising from conversational agent pipelines can be linearized into a sequential chain without loss of correctness.

Definitions. Let $G = (V, E)$ be a directed acyclic graph where $V = \{v_1, v_2, \dots, v_n\}$ are processing nodes and $E \subseteq V \times V$ are data dependencies. We assume:

- (A1) G has a unique source v_1 (no incoming edges) and a unique sink v_n (no outgoing edges).
- (A2) Every edge $(v_i, v_j) \in E$ satisfies $i < j$ (i.e., node indices already form a valid topological ordering). This is without loss of generality—nodes can always be renumbered.
- (A3) Each node’s output depends only on messages it has already received (causal processing).

Theorem. Under assumptions (A1)–(A3), G can be executed correctly by a linear chain $v_{\pi(1)} \rightarrow v_{\pi(2)} \rightarrow \dots \rightarrow v_{\pi(n)}$ connected by FIFO queues, where π is any topological ordering of G .

Proof.

1. **Existence of valid ordering.** By (A2), node indices already form a topological ordering: for every edge $(v_i, v_j) \in E$, we have $i < j$. We use this index ordering directly as the linearization order $\pi = \text{id}$.
2. **Message routing.** Arrange nodes in topological order and connect consecutive nodes by a single FIFO queue:

$$Q_0 \rightarrow v_{\pi(1)} \rightarrow Q_1 \rightarrow v_{\pi(2)} \rightarrow Q_2 \rightarrow \dots \rightarrow Q_{n-1} \rightarrow v_{\pi(n)} \rightarrow Q_n$$

When v_i produces a message destined for v_j (where $j > i$ in topological order), the message is tagged with `destination = j` and placed in Q_i .

3. **Forwarding.** Each node v_k dequeues a message from Q_{k-1} . If `destination` $\neq k$, v_k forwards the message unchanged to Q_k without processing. If `destination` $= k$, v_k processes the message.
4. **Correctness.** In the linearized pipeline, each node has exactly one input queue and processes messages one at a time. By the topological ordering guarantee, v_j will eventually dequeue the message because:
 - The message enters the chain at position $i < j$.
 - Every intermediate node v_k ($i < k < j$) forwards it because `destination` $= j \neq k$.
 - FIFO ordering within each queue preserves the temporal order of messages from the same source.

By (A3), v_j can correctly process the message because all its dependencies (from nodes earlier in the topological order) have already been processed and their outputs forwarded. For multi-dependency nodes, all predecessors are placed earlier in the linearization, so their outputs arrive at v_j in the correct order. By (A3), v_j ’s output depends only on messages it has already received, so it can accumulate inputs from predecessors via internal state and produce its output upon receiving the final dependency—no external synchronization is required.

5. **Bandwidth optimization via variable filtering.** To avoid unnecessary data transfer, each node specifies `input_vars` (fields it needs) and `pass_vars` (fields to forward). Fields not listed in either are discarded. This reduces the message size at each hop without affecting correctness.

Trade-off: serialization of parallel nodes. If v_a and v_b are topologically parallel (no path from one to the other), linearization forces one to precede the other. A message destined for v_b must wait in the queue behind v_a 's processing. The added latency is:

$$\Delta L = \sum_{k \in \text{intervening nodes}} L_k$$

where L_k is the processing time of node k for the preceding message.

For independent nodes that both process the same message, reordering does not reduce the combined latency—the total is always $\sum L_k$. The dispatcher–receiver bracket (Section 3.6) reduces this to $\max(L_k)$.

B Pipeline Configuration Example

A complete pipeline configuration for the `unity_chan` character:

Listing 1: Pipeline configuration (simplified).

```

1 {
2   "pipeline": [
3     {
4       "node_id": 1,
5       "function": "call_openai_asr",
6       "config": {
7         "input_vars": [{"input_name": "audio_file",
8                           "sources": ["audio_file"]}],
9         "output_vars": [{"output_name": "result",
10                           "targets": ["1_text"]},
11                           {"output_name": "language",
12                             "targets": ["1_language"]}],
13         "next_nodes": [3],
14         "model": "qwen_asr"
15       }
16     },
17     {
18       "node_id": 3,
19       "function": "call_openai_llm",
20       "config": {
21         "input_vars": [{"input_name": "prompt",
22                           "sources": ["1_text"]}],
23         "output_vars": [{"output_name": "text",
24                           "targets": ["3_text"]},
25                           {"output_name": "action",
26                             "targets": ["3_action"]}],
27         "next_nodes": [5],
28         "model": "qwen"
29       }
30     },
31     {
32       "node_id": 5,
33       "function": "call_data_query_link",
34       "config": {

```

```

35     "input_vars": [{"input_name": "prompt",
36                     "sources": ["3_action"]}],
37     "pass_vars": [{"sources": ["3_text"],
38                     "targets": ["5_text"]}],
39     "output_vars": [{"output_name": "result",
40                      "targets": ["5_action"]}],
41     "next_nodes": [6]
42 }
43 },
44 {
45     "node_id": 6,
46     "function": "call_openai_tts",
47     "config": {
48         "input_vars": [{"input_name": "text",
49                         "sources": ["5_text"]}],
50         "pass_vars": [{"sources": ["5_action"],
51                         "targets": ["action"]}],
52         "output_vars": [{"output_name": "audio_file",
53                          "targets": ["audio_data"]}],
54         "next_nodes": [],
55         "model": "qwen_tts"
56     }
57 }
58 ]
59 }

```

Note: `node_id` values (1, 3, 5, 6) are non-contiguous but monotonically increasing, allowing future insertion of new nodes (e.g., node 2 or 4) without renumbering existing nodes.